Layered Performance Analysis of TLS 1.3 Handshakes: Classical, Hybrid, and Pure Post-Quantum Key Exchange

David Gómez-Cambronero[1][0009−0006−4127−1853] Daniel Munteanu[2][0009−0005−1982−3706] Ana I. González-Tablas[3][0000−0002−6259−8955]

[1] Telefonica Innovación Digital, Madrid, Spain
david.gomezcambronero@telefonica.com
[2] Keysight Technologies, Bucharest, Romania
daniel.munteanu@keysight.com
[3] Universidad Carlos III de Madrid, Spain
aigonzal@inf.uc3m.es

Abstract. In this paper, we present a laboratory study focused on the impact of post-quantum cryptography (PQC) algorithms on multiple layers of stateful HTTP over TLS transactions: the TCP handshake, the intermediate TCP–TLS layer, the TLS handshake, the intermediate TLS layer, and the HTTP application layer. To this end, we propose a laboratory architecture that emulates a real-world setup in which a load test of up to 100 transactions per second is sent to a load balancer, which in turn forwards them to a backend server that returns the responses. Each set of tests is executed using the TLS 1.3 key exchange groups as follows: traditional (or non-PQC), hybrid PQC and pure PQC. Each set of tests also varied the backend response size. Across more than thirty experiments, we performed data reduction and statistical analysis for each layer, to determine the specific impact of each algorithm (PQC and traditional) at every stage of the HTTP-over-TLS transaction.

Keywords: TLS 1.3 · Performance · Post-quantum cryptography · ML-KEM · Hybrid key exchange

1 Introduction

The advent of cryptographically relevant quantum computers challenges the security assumptions of current public-key infrastructures. In response, NIST finalized its first post-quantum cryptographic (PQC) standards in August 2024: FIPS 203 (ML-KEM) [1], FIPS 204 (ML-DSA) [2], and FIPS 205 (SLH-DSA) [3]. Growing concern over "store now, decrypt later" attacks has accelerated PQC migration efforts, making it necessary to quantify performance impact prior to deployment.

While previous studies have evaluated PQC at the aggregate TLS handshake level, this work introduces a layered decomposition of the HTTP-over-TLS transaction into five protocol phases (3 typical TLS layers: TCP, TLS, App, and the

intermediate TCP-TLS and TLS-App, see Figure 1), enabling attribution of PQC-induced overhead to specific layers.

The main contributions of this paper are:

1. A five-layer latency decomposition methodology for TLS 1.3 connections, from TCP handshake through application response.
2. An empirical comparison of classical key exchange (x25519), hybrid (x25519+ML-KEM), and pure post-quantum (ML-KEM) under realistic load conditions (5 minutes with 100 TPS) in more than 30 performance tests (in total 30K requests per each performance test with a total of around 1 million requests).
3. The finding that the TLS handshake *exchange* (ClientHello→Finished) is effectively *algorithm-neutral*: all configurations—classical, hybrid, and pure ML-KEM—show negligible effect sizes (Glass's $\Delta < 0.2$–0.33), with no practically meaningful penalty or advantage attributable to the key exchange algorithm. The algorithm-sensitive cost is instead isolated to ClientHello *construction*, which we measure separately in the TCP-to-TLS layer; this finding is consistent with Sosnowski et al. [13].
4. A publicly available data-reduction tool for extracting per-layer statistics from packet captures.

The remainder of this paper is organized as follows. Section 2 reviews background and related work. Section 3 describes the methodology and experimental setup. Section 4 presents and discusses the results. Section 5 concludes and outlines future work.

2 Background and Related Work

Throughout this paper, the term *classical* (or *traditional*) refers to cryptographic algorithms that are vulnerable to attacks by quantum computers (e.g., ECDH, RSA), while *post-quantum* (PQC) refers to algorithms specifically designed to be resistant to such attacks (e.g., ML-KEM). Note that all algorithms—classical and post-quantum alike—run on conventional (non-quantum) hardware; the distinction is solely about their vulnerability to quantum adversaries [21].

2.1 PQC Standardization and Migration Context

Following the NIST PQC standards finalized in August 2024 [1,2,3], and their adoption by software providers such as the Open Quantum Safe (OQS) project [4] and OpenSSL [8], network vendors are starting to integrate these algorithms into their products. Furthermore, considering the threat of "store now, decrypt later" attacks, organizations are paying close attention to quantum-safe cryptography and building PQC transition initiatives.

The urgency of this transition is underscored by institutional mandates. The U.S. National Security Agency published updated guidance for the CNSA 2.0 algorithm suite [17], requiring all national security systems to adopt ML-KEM for key establishment by 2030 and quantum-resistant algorithms exclusively by

2033. These timelines, together with similar initiatives in the European Union and other jurisdictions, create a pressing need for empirical performance data to inform migration planning. Therefore, for all stakeholders, it is of paramount importance to understand the performance impact on their systems when migrating to PQC before pushing this into production.

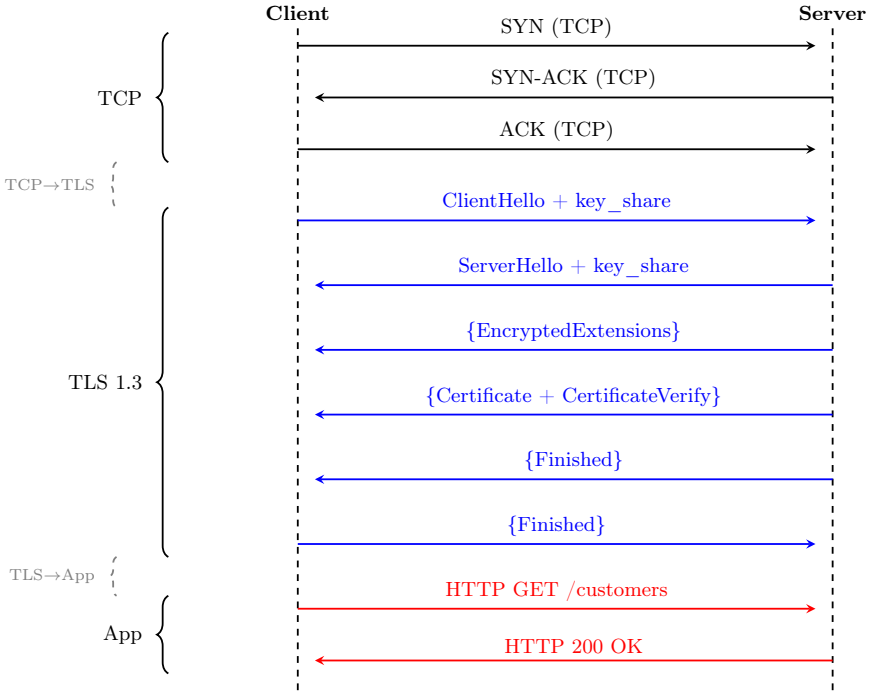

Fig. 1. TLS 1.3 full handshake over TCP with 1-RTT key exchange. Note: this study focuses on the 1-RTT full handshake mode (no 0-RTT early data, no session resumption) to measure the unmitigated impact of PQC on initial connection establishment.

2.2 TLS 1.3

TLS 1.3 (RFC 8446) is the latest version of the Transport Layer Security protocol, designed to provide confidentiality, integrity, and authentication for Internet communications. Compared to its predecessors, TLS 1.3 reduces the handshake to a single round trip (1-RTT), removes legacy cipher suites, and mandates forward secrecy through ephemeral key exchange. In a typical TLS 1.3 full handshake, the connection proceeds through three main protocol phases: TCP establishment, TLS negotiation, and application data exchange, each of which can be further decomposed into sub-phases. Figure 1 illustrates this decomposition, showing the five layers defined in this study: the TCP handshake,

the intermediate TCP-to-TLS delay, the TLS handshake itself, the intermediate TLS-to-Application delay, and the application response.

The rationale for decomposing the end-to-end latency into these five distinct layers is that each protocol phase is affected differently by PQC. Specifically: (i) the **TCP handshake** (SYN to SYN-ACK) serves as a control layer—it should be independent of the cryptographic algorithm and any variation here indicates system-level effects; (ii) the **TCP-to-TLS delay** (SYN-ACK to ClientHello) captures the client-side cost of TLS context initialization and key material generation, which is where PQC is expected to have the largest impact; (iii) the **TLS handshake exchange** (ClientHello→Finished) reflects the on-the-wire cost of key exchange, certificate transmission, and cryptographic verification; (iv) the **TLS-to-Application delay** (Finished→ HTTP GET) isolates any residual overhead after secure channel establishment; and (v) the **Application response** (HTTP GET→HTTP 200 OK) measures the client-to-backend response time, which should be algorithm-independent.

Note that the **TCP-to-TLS delay** is not a networking layer in the traditional protocol-stack sense. During this interval the client initializes the TLS context, generates the ephemeral key-exchange material for every group offered in the ClientHello, and serializes the ClientHello. This distinction is essential for interpreting the results: when we later describe the TLS handshake exchange as *algorithm-neutral*, we refer specifically to the ClientHello→Finished message exchange, not to the ClientHello *construction* cost, which is measured separately in the TCP-to-TLS delay.

By analyzing each layer separately, we can precisely attribute the PQC overhead to the specific protocol phase where it originates, rather than observing only the aggregate effect on end-to-end latency.

2.3 PQC Deployment in TLS

The integration of post-quantum algorithms into production TLS stacks has progressed rapidly. Google Chrome enabled hybrid post-quantum key exchange by default in Chrome 124 (April 2024) [5]. Cloudflare reported large-scale deployment and operational considerations for hybrid TLS in production [6]. Amazon Web Services published production support updates for post-quantum TLS in AWS KMS and s2n-tls [7]. On the open-source side, the OQS project [4] provides liboqs and an OpenSSL provider that enables PQC algorithm negotiation in standard TLS stacks—the same toolchain used in this study.

Paquin et al. [18] conducted systematic benchmarks of PQC algorithms integrated into the OQS-OpenSSL fork, measuring handshake completion times and throughput for various KEM and signature combinations. Their work provides valuable computational baselines but evaluates aggregate handshake times without per-layer decomposition and predates the final NIST standards.

2.4 Performance Studies and contribution of this work

Several performance studies have addressed the impact of post-quantum algorithms on TLS. Sikeridis et al. [11] focus on post-quantum digital signatures and provide a comparative analysis of TLS authentication overhead. Similarly, Raabi et al. [12] evaluate post-quantum signature schemes in isolation.

Sosnowski et al. [13] compare TLS handshake latency across several pre- and post-quantum algorithms, reporting aggregate handshake times without per-layer decomposition. Kampanakis et al. [14] evaluate the Time-To-Last-Byte (TTLB) between P-256 and post-quantum algorithms using real-world connection data. Zheng et al. [15] demonstrate that ML-KEM can achieve competitive performance despite larger key material. Liu et al. [22] report on the PQC migration of a physical 5G testbed using ML-KEM 768 (hybrid with x25519) for TLS, IPSec, and SUCI; their evaluation shows minimal performance degradation (throughput drop <4.3%, latency variation within ±0.3 ms), consistent with our finding that PQC overhead is concentrated in connection establishment and negligible at steady state.

In contrast to these works, our study (i) decomposes the transaction into five protocol layers rather than measuring only end-to-end latency, (ii) compares classical, hybrid, and pure PQC configurations side by side, and (iii) demonstrates that the TLS handshake *exchange* (ClientHello→Finished) is effectively *algorithm-neutral*—with the algorithm-sensitive cost isolated to ClientHello construction in the TCP-to-TLS layer—with all PQC configurations showing negligible effect sizes (Glass's $\Delta < 0.33$) [25,24]—a granularity of analysis not reported in prior literature. Note that hybrid key exchange (e.g., x25519_MLKEM768) follows the IETF design draft [9] and an active ECDHE-MLKEM specification track [10], and is not yet a finalized RFC, unlike the pure ML-KEM algorithms standardized in FIPS 203 [1].

To make the contribution boundaries explicit, Table 1 positions prior studies against this work along four objective, verifiable criteria: scope (key exchange vs. signature), metric granularity (aggregate vs. per-layer), test methodology, and whether backend response size is varied as an independent variable.

Table 1. Feature comparison with related work along objective criteria.

Study	Scope	Granularity	Test model	Response var.
Paquin et al. (2020) [18]	KEx + Sig	Aggregate	Microbenchmark	—
Sikeridis et al. (2020) [11]	Signature	Aggregate	Emulated	—
Raabi et al. (2023) [12]	Signature	Aggregate	Emulated	—
Sosnowski et al. (2024) [13]	KEx	Aggregate	Internet-scale	—
Kampanakis et al. (2024) [14]	KEx	Aggregate	Real-world	—
Zheng et al. (2024) [15]	KEx	Aggregate	Emulated	—
Liu et al. (2025) [22]	KEx + IPSec	Aggregate	Physical 5G	—
This work	**KEx**	**5-layer**	**Emulated (100 TPS)**	✓

3 Methodology and Experimental Setup

3.1 Objective

The objective of this study is to evaluate the performance impact of integrating post-quantum cryptography (PQC) into the TLS 1.3 protocol. The experiments focus on handshake latency, message size, and connection throughput under realistic network conditions, comparing classical, hybrid, and post-quantum configurations.

3.2 Environment Configuration

Server side (Nginx reverse proxy). To simulate the typical configuration in several production environments, we implemented a TLS-terminating endpoint using **Nginx 1.27.3**, compiled from source against a custom build of **OpenSSL 3.4.0** integrated with **liboqs 0.12.0** and the **OQS provider 0.8.0**. The base image is Alpine 3.21. Server certificates are signed with **RSA-2048** (`rsa:2048`). The supported key exchange groups are configured in the OpenSSL provider configuration and include: x25519, x448, prime256v1, secp384r1, mlkem512, mlkem768, mlkem1024, X25519MLKEM512, X25519MLKEM768, and SecP256r1MLKEM768. The system OpenSSL libraries were replaced by the custom build to avoid conflicts.

Client side (Keysight CyPerf). Traffic generation was performed using **Keysight CyPerf** [16], a high-performance stateful traffic generator that emulates high-scale HTTPS clients and servers (with up to hundreds of thousands of TLS handshakes per second). CyPerf agents have built-in native support for ML-KEM key exchange groups, enabling negotiation of both classical and post-quantum algorithms during the TLS 1.3 handshake. The client workloads consist of concurrent virtual users executing stateful HTTP transactions at a sustained rate of 100 TPS to emulate realistic service load (well below the traffic generation tool's capability).

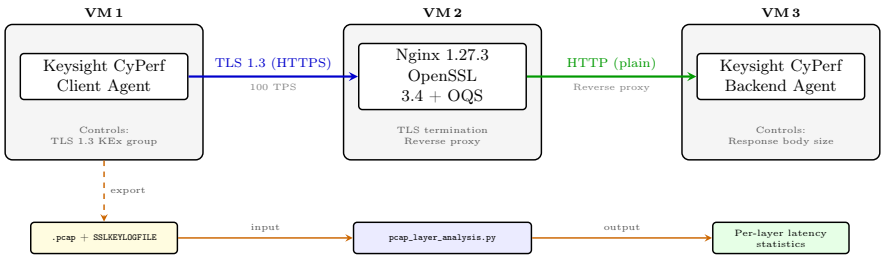

Fig. 2. Test bed architecture. Three VMs connected via local networking: the CyPerf client agent injects HTTPS traffic at 100 TPS towards Nginx (TLS termination with OQS), which proxies plain HTTP to the CyPerf backend agent. The key exchange group and response body size are independently controlled by CyPerf. The captured `pcap` files and TLS key logs are processed by the data-reduction script to extract per-layer latency statistics.

3.3 Metrics and Data Collection

Performance evaluation is organized into three metric families:

Per-layer latency metrics. For each of the five protocol layers described in Section 2.2, the data-reduction tool computes the following descriptive statistics from all valid TCP streams in a capture: arithmetic mean, median (p50), p90, p95, and p99 percentiles, minimum, maximum, and standard deviation (SD). Additionally, the **total end-to-end time (E2E)**, defined as the per-connection wall-clock interval from the TCP SYN to the final HTTP 200 OK, is computed per connection and summarized with its mean and SD. The E2E time is obtained directly from the per-connection boundary timestamps (not by summing per-layer percentiles, which are not additive).

Message size and key material metrics. For each algorithmic configuration, the `ClientHello` length (bytes) and `ServerHello` length (bytes) are recorded, as well as the client key exchange (key_share) payload size. The `key_share` payload size is deterministic for a given key exchange group, as it is fixed by the algorithm specification [1]. This allows direct quantification of the bandwidth overhead introduced by PQC key material.

Resource utilization metrics. CPU utilization is recorded separately for client (CyPerf agent) and server (Nginx) VMs, broken down into *System*, *User*, and *Idle* components. Network throughput (Mb/s) is captured on the server-facing interface (eth1 recv). These metrics enable correlation between cryptographic workload and system resource consumption.

Test scenarios. Three backend response configurations are evaluated per algorithm: (i) *direct* (minimal response body), (ii) *4 KB response body*, and (iii) *40 KB response body*. Here the *response body size* refers to the length of the HTTP payload returned by the backend server (and relayed by the Nginx reverse proxy) to the client in the application-data TLS records that follow handshake

completion that is, the bytes delivered after the secure channel is established, not part of the handshake itself. This variation isolates the cryptographic overhead from the application-layer transfer cost.

Each configuration is executed multiple times to ensure statistical reliability.

Effect-size criterion. To assess whether observed latency differences are practically meaningful, we compute Glass's Δ [25]: the difference between the PQC and x25519 means, divided by the standard deviation (SD) of the x25519 baseline, which represents the inherent measurement variability without PQC influence. We interpret $|\Delta|$ following Cohen's conventional thresholds [24]: $|\Delta| < 0.2$ (negligible), 0.2–0.8 (small to medium), and $|\Delta| > 0.8$ (large).

3.4 Data Reduction from Packet Captures

To perform the layer-by-layer latency decomposition, full packet captures (`pcap` files) were recorded during each load test using `CyPerf`. A key requirement of this methodology is that, for *each* performance-test run, TLS key material (session/master keys) is exported through `SSLKEYLOGFILE`; without this step, packet-level decryption and reliable request-level correlation across TCP/TLS/HTTP boundaries is not possible. CyPerf is able to export the `SSLKEYLOGFILE` as well, together with the associated packet capture. These captures were then processed by a custom Python script pcap_layer_analysis [23] that decrypts and parses each TCP stream to extract per-connection timestamps at the following protocol boundaries: SYN, SYN-ACK, ClientHello, TLS Finished, HTTP GET, and HTTP 200 OK. For each connection, the tool computes the five inter-layer deltas and aggregates them into percentile-based statistics (p50, p90, p95, p99) across all streams in the capture file, enabling the statistical analysis presented in this paper. The tool supports parallel processing of large captures by splitting them into stream batches via `tshark`. The full repository and the script used for this step are publicly available [23].

3.5 Algorithmic Configurations

The experiments compare several algorithmic combinations for TLS 1.3 key exchange. In all configurations, the server authentication uses **rsa:2048** certificates, generated by the OQS-enabled OpenSSL build. Since the signature algorithm is held constant across all tests, the measured latency differences are attributable exclusively to the key exchange algorithm. The key exchange groups tested are:

- **Classical**: x25519 (Curve25519 ECDH).
- **Hybrid**: x25519_MLKEM512, x25519_MLKEM768 (combining classical ECDH with ML-KEM as defined in the IETF hybrid key exchange draft [9]).
- **Pure Post-Quantum**: MLKEM512, MLKEM1024 (ML-KEM only, per FIPS 203 [1]).
- Additional tests varying the digital signature algorithm (e.g., ECDSA vs. ML-DSA vs. SLH-DSA) to isolate signature overhead are planned as future work.

3.6 Monitoring and Visualization

All metrics are exported through **Prometheus** and visualized using **Grafana** and **CyPerf**, enabling percentile-based analysis of TLS handshake and data transfer times. This monitoring infrastructure supports long-duration experiments and simplifies the identification of bottlenecks in PQC-enabled TLS implementations.

3.7 Reproducibility and Limitations

To ensure reproducibility, the entire experimental setup is defined through containerized deployment manifests and Dockerfiles stored in a public Git repository. All containers are pinned to specific base images and software versions, including:

- **Base image**: Alpine 3.21.
- **OpenSSL**: 3.4.0 (built from source from the official repository).
- **liboqs**: 0.12.0 (Open Quantum Safe library for PQC primitives).
- **OQS provider**: 0.8.0 (OpenSSL provider enabling PQC algorithms).
- **Nginx**: 1.27.3, compiled with `-with-openssl` pointing to the custom OpenSSL build.
- **Server certificate signature**: rsa-2048 (`rsa:2048`).
- **Keysight CyPerf**: with standard, native ML-KEM support.

The main limitations of this study are:

- The use of virtualized networking may slightly reduce absolute throughput compared to bare-metal environments.
- Only single-node Docker deployments were evaluated; distributed scenarios with inter-node latency were left for future work.
- All tests employ TLS 1.3 **1-RTT full handshake** mode with fresh connections (no session resumption, no 0-RTT early data). While 0-RTT would allow sending application data (HTTP GET) in the first client flight—thus significantly reducing perceived latency—this mode was deliberately excluded to isolate and measure the full cryptographic overhead of each algorithm without amortization. Production deployments may achieve lower latencies through session resumption or 0-RTT, but the PQC-induced overhead measured here represents the unavoidable cost for initial connections.
- The packet capture was performed in software, hence additional software processing layers might introduce extra delays.

4 Results and Discussion

This section presents and discusses the experimental results obtained from the layer-by-layer analysis of TLS 1.3 connections under classical, hybrid, and post-quantum cryptographic configurations. Latency results are reported in milliseconds, with percentile-based metrics emphasized to mitigate the influence of sporadic outliers. The analysis is organized into the following subsections:

- **Section 4.1** provides an overview of the per-layer latency decomposition, summarizing the full dataset in a single reference table and a stacked visualization.
- **Section 4.2** analyzes the TCP handshake layer, confirming its independence from the cryptographic algorithm.
- **Section 4.3** examines the TCP-to-TLS delay, where the dominant PQC overhead is localized.
- **Section 4.4** evaluates the TLS handshake exchange latency, demonstrating its algorithm-neutral behavior.
- **Section 4.5** covers post-handshake and application-layer latency, showing negligible PQC impact.
- **Section 4.6** introduces normalized metrics (Overhead Factor and Cryptographic Overhead Share) to isolate and quantify the PQC penalty.
- **Section 4.7** evaluates the effect of increasing the backend response payload from 4 KB to 40 KB.
- **Section 4.8** reports CPU and network utilization across configurations.
- **Section 4.9** consolidates the end-to-end perspective, combining per-layer results into aggregate connection times.

4.1 Latency Breakdown by Protocol Layer

Table 2 summarizes the observed latency percentiles for the different protocol layers under the backend 4 KB response body, along with 40 KB response tests for x25519 and x25519_MLKEM768 to assess PQC impact on larger payloads (analyzed in Section 4.7). Each layer is defined by its start and end timestamps in the TLS connection lifecycle. Five representative configurations are compared:

Table 2. Latency percentiles and standard deviation per protocol layer. Rows labeled (4 KB) and (40 KB) refer to the backend response body size. Comparing x25519 (classical), x25519_MLKEM512 (hybrid), x25519_MLKEM768 (hybrid), MLKEM512 (pure PQC), and MLKEM1024 (pure PQC). The 40 KB scenario is included for x25519 and x25519_MLKEM768 to support the analysis in Section 4.7.

Layer (Timestamp Start → End)	p50 (ms)	p95 (ms)	p99 (ms)	SD (ms)
TCP handshake (SYN → SYN-ACK)				
x25519 (4 KB)	0.360	0.694	0.957	0.248
x25519_MLKEM512 (4 KB)	0.390	3.060	4.147	0.999
x25519_MLKEM768 (4 KB)	0.402	2.720	4.139	1.016
MLKEM512 (4 KB)	0.381	2.882	3.892	0.969
MLKEM1024 (4 KB)	0.393	2.397	3.901	0.915
x25519 (40 KB)	0.401	0.781	1.251	16.993
x25519_MLKEM768 (40 KB)	0.417	3.386	5.048	10.163
TCP-to-TLS delay (SYN-ACK → ClientHello)				
x25519 (4 KB)	0.294	0.635	0.800	0.194
x25519_MLKEM512 (4 KB)	1.866	4.122	5.664	1.196
x25519_MLKEM768 (4 KB)	1.903	3.900	5.534	1.411
MLKEM512 (4 KB)	1.726	3.577	5.043	1.065
MLKEM1024 (4 KB)	1.772	3.608	5.088	1.044
x25519 (40 KB)	0.289	0.702	0.901	0.232
x25519_MLKEM768 (40 KB)	1.915	4.097	5.598	1.421
TLS handshake (ClientHello → Finished)				
x25519 (4 KB)	5.547	10.903	12.697	2.893
x25519_MLKEM512 (4 KB)	5.879	11.296	13.256	2.871
x25519_MLKEM768 (4 KB)	6.495	11.692	13.650	2.982
MLKEM512 (4 KB)	5.253	9.999	12.010	3.798
MLKEM1024 (4 KB)	5.450	10.578	12.209	2.701
x25519 (40 KB)	5.791	10.886	12.242	2.827
x25519_MLKEM768 (40 KB)	6.105	10.715	12.603	7.055
TLS-to-Application delay (Finished → HTTP GET)				
x25519 (4 KB)	0.526	1.227	1.912	0.389
x25519_MLKEM512 (4 KB)	0.991	2.363	3.479	0.747
x25519_MLKEM768 (4 KB)	1.004	2.576	3.730	0.976
MLKEM512 (4 KB)	0.897	2.605	3.647	1.741
MLKEM1024 (4 KB)	0.950	2.536	3.554	0.635
x25519 (40 KB)	0.559	1.272	2.061	0.415
x25519_MLKEM768 (40 KB)	0.989	2.546	3.741	0.849
Application response (HTTP GET → HTTP 200 OK)				
x25519 (4 KB)	9.071	14.866	17.424	3.553
x25519_MLKEM512 (4 KB)	8.880	14.925	17.510	3.544
x25519_MLKEM768 (4 KB)	8.334	14.186	16.868	3.375
MLKEM512 (4 KB)	9.087	15.099	17.802	3.581
MLKEM1024 (4 KB)	8.838	14.442	17.026	3.451
x25519 (40 KB)	12.032	18.935	22.169	25.467
x25519_MLKEM768 (40 KB)	11.166	18.479	21.721	16.792

Figure 3 provides a visual decomposition of the end-to-end latency by protocol layer at both the median (p50) and tail (p95) percentiles for each algorithm under the 4 KB backend scenario.

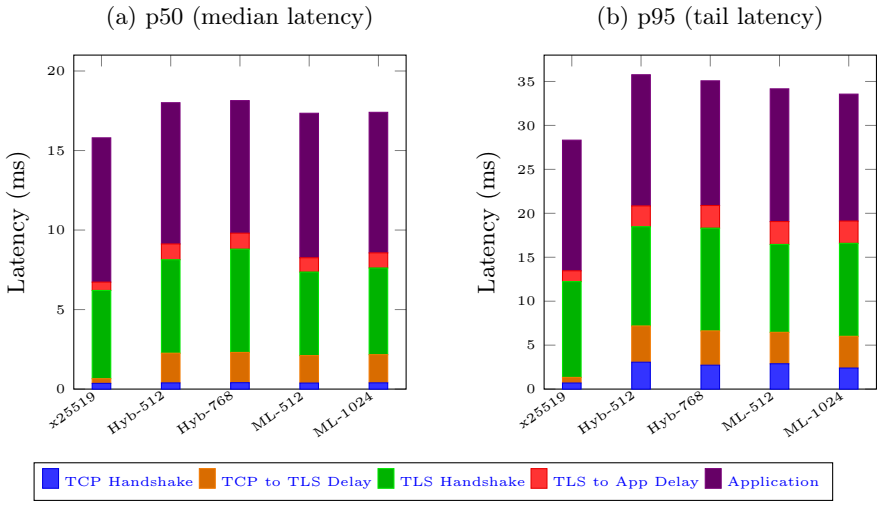

Fig. 3. End-to-end latency decomposition by protocol layer (backend 4 KB). Left: p50 (median). Right: p95 (tail). Abbreviations: Hyb-512 = x25519_MLKEM512, Hyb-768 = x25519_MLKEM768, ML-512 = MLKEM512, ML-1024 = MLKEM1024.

Comparing (a) and (b) reveals that the TCP and TCP→TLS layers grow disproportionately at the tail under PQC, while TLS and application layers scale proportionally. Additional tests with a 40 KB backend response body were also conducted for x25519 and x25519_MLKEM768; the corresponding analysis is presented in Section 4.7

4.2 TCP Handshake Latency

The TCP handshake latency results in Table 2 show no statistically significant dependency on the cryptographic algorithm at the median: values range from 0.360 to 0.402 ms across all configurations, a maximum difference of 0.042 ms ($\Delta = 0.17$, negligible per Cohen's thresholds; Section 3.3). This confirms that the TCP layer is genuinely algorithm-independent. However, at p95, tail latencies increase under PQC configurations (up to 3.06 ms for x25519_MLKEM512 vs. 0.694 ms for x25519)—an effect attributable to higher system load rather than cryptographic overhead, since the TCP layer precedes any key exchange computation.

4.3 Client-Side TLS Initialization Cost

The TCP-to-TLS delay (SYN-ACK to ClientHello) captures the cost of TLS context initialization and ClientHello construction. This layer exhibits the most pronounced post-quantum impact, aligned with real-world expectations, with median latency increasing from approximately 0.3 ms for x25519 to around 1.8–1.9 ms for both hybrid and pure post-quantum configurations (see Table 2).

This increase is consistent across MLKEM variants and independent of backend response size, indicating that the dominant cost arises from client-side initialization and serialization.

Because this cost is incurred entirely before the first byte is placed on the wire, it is also a prime candidate for optimization. A TLS library could *precompute* the ephemeral ClientHello key material ahead of connection establishment—for example, by maintaining a small pool of ready-to-use key shares for the offered groups—moving post-quantum key generation off the connection critical path. Such precomputation would render the TCP-to-TLS overhead factor effectively negligible without altering the on-the-wire protocol, since the heavy operation (ephemeral key-pair generation) is independent of the peer and can be performed in advance.

Figure 4 presents the full percentile profile (p50, p90, p95, p99) for this layer, revealing that the 6× overhead at the median is preserved across all percentiles—a strong indicator that the latency shift is consistent rather than driven by outliers.

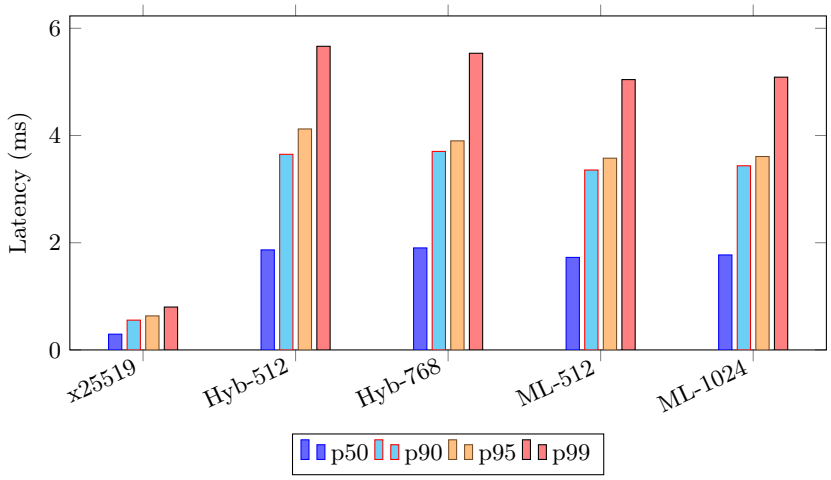

Fig. 4. TCP-to-TLS delay (SYN-ACK to ClientHello): full percentile profile per algorithm (backend 4 KB). Abbreviations: Hyb-512 = x25519_MLKEM512, Hyb-768 = x25519_MLKEM768, ML-512 = MLKEM512, ML-1024 = MLKEM1024.

4.4 TLS Handshake Latency

The TLS handshake latency (ClientHello to Finished) reveals a nuanced picture. **Hybrid** configurations (x25519_MLKEM512, x25519_MLKEM768) show median values of 5.879 ms and 6.495 ms respectively, compared to 5.547 ms for x25519. However, these differences (0.332–0.948 ms) must be evaluated against the baseline standard deviation of 2.893 ms: Glass's $\Delta = 0.11$–0.33, well below Cohen's negligible threshold of 0.2 for the smaller hybrid and negligible-to-small for x25519_MLKEM768 (Section 3.3), indicating that the effect is *practically negligible*. Similarly, **pure ML-KEM** variants show slightly lower median latencies (MLKEM512: 5.253 ms, MLKEM1024: 5.450 ms), but the differences (-0.097 to -0.294 ms, $|\Delta| = 0.03$–0.10) are negligible and cannot be attributed to a genuine computational advantage. The observation that ML-KEM operations are inherently efficient—relying on parallel polynomial arithmetic over structured lattices rather than sequential scalar multiplication [20,18,19]—is well-documented in microbenchmarks, but at the TLS protocol level the differences are absorbed by protocol framing, network jitter, and system scheduling, making them indistinguishable from the baseline in our measurements.

Figure 5 illustrates the TLS handshake latency for p50, p95, and p99 across all evaluated algorithms, graphically supporting the previous observation that the differences remain small and within measurement noise.

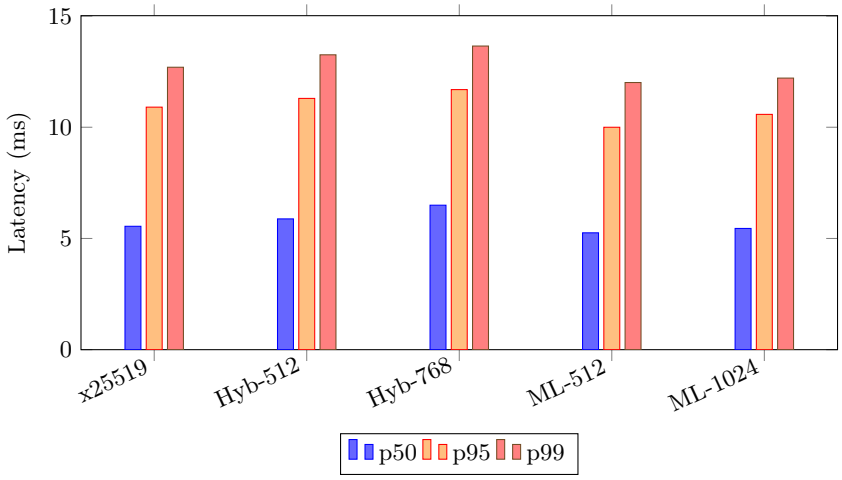

Fig. 5. TLS handshake latency (ClientHello to Finished) per algorithm (backend 4 KB). Abbreviations: Hyb-512 = x25519_MLKEM512, Hyb-768 = x25519_MLKEM768, ML-512 = MLKEM512, ML-1024 = MLKEM1024.

4.5 Post-Handshake and Application Latency

The TLS-to-application delay reflects residual effects immediately after secure channel establishment. Median latency increases from 0.526 ms (x25519, SD = 0.389 ms)

to 0.897–1.004 ms for post-quantum configurations. Glass's $\Delta = 0.95$–1.23, which Cohen's scale classifies as a large effect size (Section 3.3). However, the absolute magnitude is small (0.37–0.48 ms), stable across configurations, and independent of backend response size, suggesting that while statistically detectable, the practical impact is limited.

Application response latency is dominated by end-to-end network path latency and response size. Backend responses with a 4 KB body increase latency to approximately 8–9 ms. Across all cryptographic algorithms, the maximum deviation from the x25519 baseline is 0.74 ms (overall range 8.334–9.087 ms, span 0.75 ms), yielding $\Delta = 0.21$—negligible to small per Cohen's thresholds (Section 3.3). This confirms that no practically meaningful application-level penalty attributable to post-quantum cryptography is observed. The effect of larger response payloads (40 KB) on the PQC overhead is analyzed in Section 4.7.

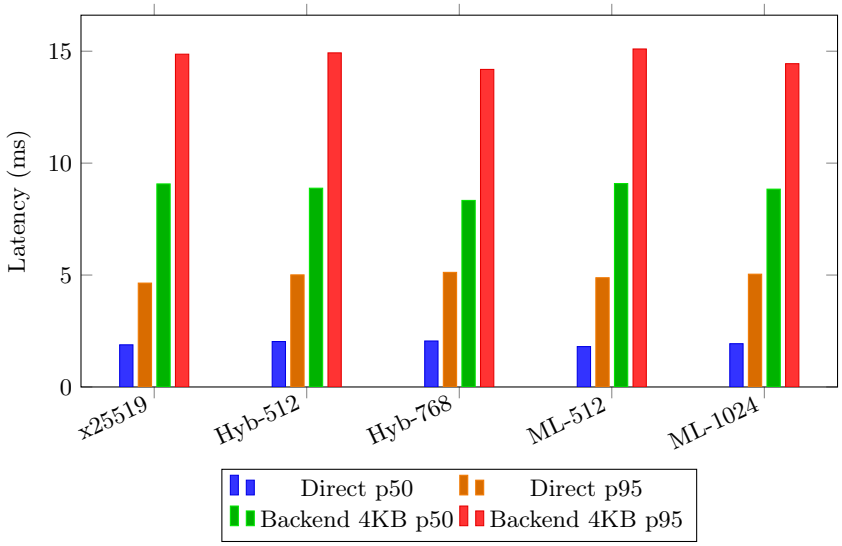

Fig. 6. Application response latency per algorithm and scenario. Within each group, the near-identical bar heights confirm that PQC has no measurable impact at the application layer, at either percentile. Abbreviations: Hyb-512 = x25519_MLKEM512, Hyb-768 = x25519_MLKEM768, ML-512 = MLKEM512, ML-1024 = MLKEM1024.

4.6 Normalized Cryptographic Overhead

The preceding per-layer analysis reveals that only the TCP-to-TLS and TLS handshake layers exhibit measurable cryptographic sensitivity, while TCP, TLS-to-App, and application layers behave as algorithm-independent. To isolate and quantify this impact more precisely, we now normalize the overhead of these two crypto-sensitive layers relative to the classical baseline, providing a clearer measure of the actual PQC cost within the end-to-end transaction.

We define two complementary metrics. The **Overhead Factor (OF)** captures the per-layer slowdown relative to the classical baseline:

$$\mathrm{OF}^{(l)} = \frac{L^{(l)}_{\mathrm{PQC}}}{L^{(l)}_{\mathrm{x25519}}} \tag{1}$$

where $L^{(l)}$ denotes the latency at a given percentile for protocol layer l. An OF of 1.0 indicates no overhead; values below 1.0 indicate a speedup.

The **Cryptographic Overhead Share (COS)** aggregates the overhead of the two cryptographic-sensitive layers—TCP-to-TLS and TLS handshake—and expresses it as a fraction of the total end-to-end connection time:

$$\mathrm{COS} = \frac{\sum_{l \in \{\text{TCP-to-TLS, TLS}\}} \left(L^{(l)}_{\mathrm{PQC}} - L^{(l)}_{\mathrm{x25519}}\right)}{L^{(\mathrm{e2e})}_{\mathrm{PQC}}} \times 100\% \tag{2}$$

COS quantifies what fraction of the total PQC connection time is attributable *exclusively* to post-quantum cryptographic overhead, isolating it from protocol and application contributions. A COS of 0% means no cryptographic penalty; higher values indicate a larger share of connection time consumed by PQC.

Table 3 and Figure 7 present these metrics for all PQC configurations at the median (p50) and tail (p95) percentiles.

Table 3. Normalized cryptographic overhead per algorithm. OF: Overhead Factor (Eq. 1); COS: Cryptographic Overhead Share (Eq. 2). Baseline x25519 has OF = 1.00 and COS = 0% by definition.

Algorithm	$\mathbf{OF}_{\text{TCP-to-TLS}}$	$\mathbf{OF}_{\text{TLS-HS}}$	$\mathbf{OF}_{\text{Combined}}{}^{*}$	**COS (%)**
p50 (median)				
x25519	1.00	1.00	1.00	—
x25519_MLKEM512	6.35	1.06	1.33	10.6
x25519_MLKEM768	6.47	1.17	1.44	14.1
MLKEM512	5.87	0.95	1.19	6.6
MLKEM1024	6.03	0.98	1.24	7.9
p95 (tail latency)				
x25519	1.00	1.00	1.00	—
x25519_MLKEM512	6.49	1.04	1.34	10.8
x25519_MLKEM768	6.14	1.07	1.35	11.6
MLKEM512	5.63	0.92	1.18	6.0
MLKEM1024	5.68	0.97	1.23	7.9

$^{*}\mathrm{OF}_{\text{Combined}} = (L^{\text{PQC}}_{\text{TCP-to-TLS}} + L^{\text{PQC}}_{\text{TLS}})/(L^{\text{x25519}}_{\text{TCP-to-TLS}} + L^{\text{x25519}}_{\text{TLS}})$

Key findings. The TCP-to-TLS layer consistently exhibits an overhead factor ($\mathrm{OF}_{\text{TCP-to-TLS}}$, Table 3) of approximately 6× across all PQC variants and

percentiles. This overhead is unambiguously significant: Glass's $\Delta \approx 7.7$ (absolute difference $\sim$1.5 ms vs. baseline SD of 0.194 ms), far exceeding Cohen's large-effect threshold of 0.8 (Section 3.3), confirming that client-side key material generation is the dominant and systematic source of PQC latency. In contrast, the TLS handshake exchange overhead factor ($OF_{TLS\text{-}HS}$) remains close to 1.0: hybrid algorithms show a nominal increase (1.04–1.17×) and pure ML-KEM variants show $OF_{TLS\text{-}HS}$ values nominally *below* 1.0 (0.92–0.98×). However, these TLS handshake differences correspond to $\Delta = 0.03$–0.33 (absolute deltas of 0.1–0.9 ms against a baseline SD of 2.893 ms), well below Cohen's small-effect threshold of 0.5, meaning they are *practically negligible*. The data is therefore consistent with the TLS handshake exchange being effectively *algorithm-neutral* at this load level, rather than demonstrating a clear speedup or slowdown.

The COS metric reveals that, despite the large per-layer overhead in TCP-to-TLS, the cryptographic penalty accounts for only 6–14% of the total end-to-end connection time. Hybrid configurations exhibit the highest COS (10.6–14.1%), as they combine ECDH and KEM costs, whereas pure ML-KEM configurations achieve a lower COS (6.0–7.9%). Note, however, that the COS difference between hybrid and pure ML-KEM configurations is driven primarily by the $OF_{TLS\text{-}HS}$ values, which—as established above—are not statistically distinguishable from 1.0. The robust conclusion is that the overall COS is moderate (below 15%) for all PQC configurations tested.

On the relationship between computational cost and security level. Although the $OF_{TLS\text{-}HS}$ values for pure ML-KEM (0.92–0.98×) fall below 1.0, we have shown that these differences are not statistically significant at the TLS protocol level.

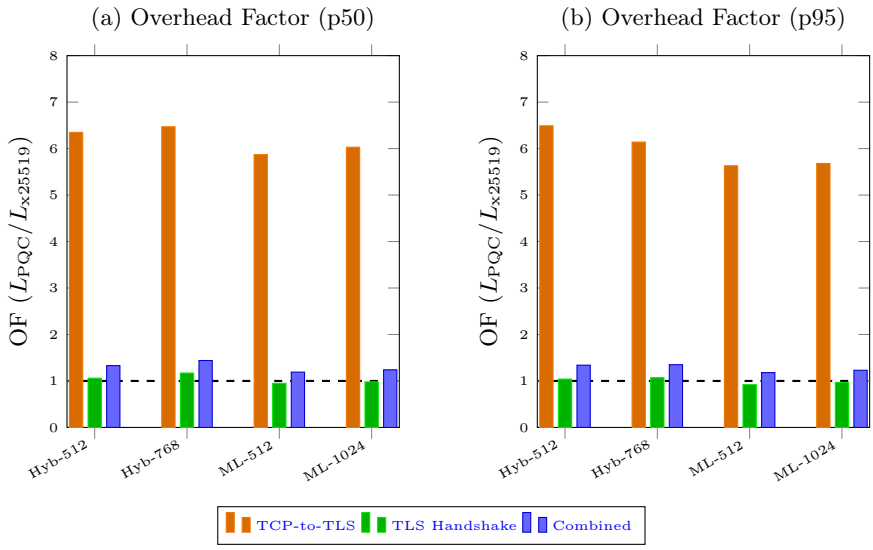

Fig. 7. Overhead Factor per algorithm for the two cryptographic-sensitive layers and their combination. The dashed line at OF = 1 marks the x25519 baseline. Left: p50. Right: p95. The TCP-to-TLS layer dominates the overhead (∼6×, $\Delta \approx 7.7$, far exceeding Cohen's large-effect threshold). $OF_{TLS\text{-}HS}$ values remain near 1.0 for all configurations; $\Delta = 0.03$–0.33, negligible per Cohen's thresholds (Section 3.3).

4.7 Impact of Response Payload Size on PQC Overhead

To evaluate whether the cryptographic overhead introduced by post-quantum algorithms scales with application-layer payload size, additional tests were conducted using a 40 KB backend response body for x25519 (classical) and x25519_MLKEM768 (hybrid). These results can be compared directly against their 4 KB counterparts reported in previous sections. Table 2 includes the full per-layer latency breakdown for both payload sizes, and Figure 8 provides a stacked visualization of the end-to-end latency decomposition across the four configurations.

TCP Handshake layer unaffected. The TCP layer show no statistically meaningful variation between the 4 KB and 40 KB scenarios for either algorithm.

Application layer dominance. Increasing the response body from 4 KB to 40 KB raises the median application response latency from 9.07 ms to 12.03 ms for x25519 (+32.7%) and from 8.33 ms to 11.17 ms for x25519_MLKEM768 (+34.0%). This increase is attributable entirely to the larger payload transfer and end-to-end network path delay; no interaction with the cryptographic algorithm is observed.

Cryptographic layers unaffected. The TCP-to-TLS delay, TLS handshake, and TLS-to-application layers show no statistically meaningful variation between the 4 KB and 40 KB scenarios for either algorithm. For x25519_

MLKEM768, the median TCP-to-TLS delay is 1.903 ms (4 KB) versus 1.915 ms (40 KB); the TLS handshake latency is 6.495 ms versus 6.105 ms; and the TLS-to-application delay is 1.004 ms versus 0.989 ms. These differences are within measurement noise and confirm that the PQC overhead is entirely *front-loaded* during connection establishment and independent of subsequent data transfer volume.

Dilution of relative PQC overhead. From an end-to-end perspective, the absolute PQC overhead (the difference between x25519_MLKEM768 and x25519) remains approximately constant: 2.34 ms at the median for the 4 KB scenario versus 1.52 ms for the 40 KB scenario. However, because the total connection time increases with payload size, the *relative* overhead decreases from approximately 14.8% (4 KB) to 8.0% (40 KB) at the median. At p95, the absolute overhead is 6.75 ms (4 KB) versus 6.65 ms (40 KB), while the relative overhead drops from 23.8% to 20.4%. This dilution effect implies that in real-world applications with substantial response payloads, the performance penalty of PQC migration becomes proportionally less significant.

Scope of payload analysis. The 40 KB evaluation was conducted only for x25519 and x25519_MLKEM768 (hybrid). Extending this analysis to pure MLKEM configurations (MLKEM512, MLKEM1024) with larger payload sizes is planned for future work.

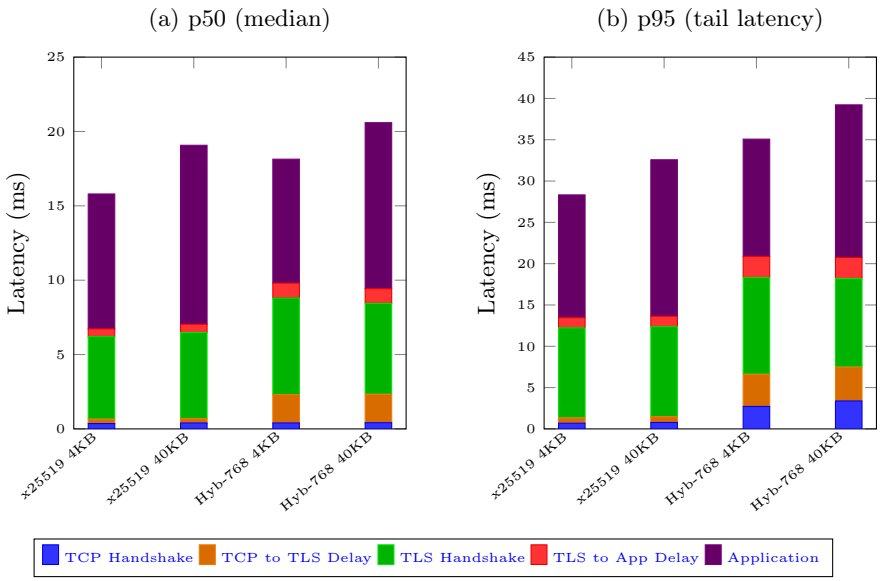

Fig. 8. End-to-end latency decomposition comparing 4 KB and 40 KB backend response bodies for x25519 (classical) and x25519_MLKEM768 (hybrid). Left: p50. Right: p95.

The cryptographic layers (TCP-to-TLS, TLS handshake, TLS-to-App) remain essentially unchanged with increasing payload size, while the application layer grows proportionally. The relative PQC overhead diminishes as the payload-dependent portion of the total latency increases.

4.8 CPU and Network Utilization

Table 4 summarizes CPU utilization and network overhead in cryptographic configurations, while Figure 9 depicts graphically the CPU utilization .

Table 4. Mean CPU utilization and network overhead per cryptographic configuration (backend 4 KB). Client and Server CPU show total utilization (System + User).

Algorithm	Client CPU (%)	Server CPU (%)	Traffic (Mb/s)	key_share (B)
x25519	3.72	14.61	5.18	32
x25519_MLKEM512	8.24	15.38	5.88	832
x25519_MLKEM768	8.71	15.46	6.14	1216
MLKEM512	7.91	13.91	5.83	800
MLKEM1024	7.82	14.34	6.55	1568

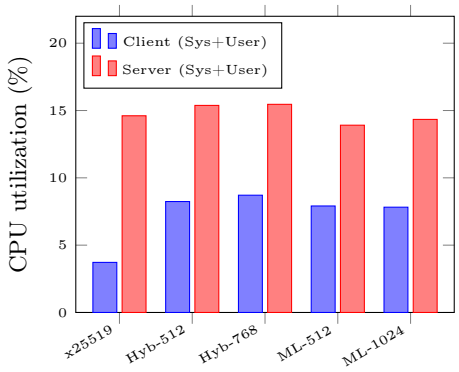

Fig. 9. Total CPU utilization (Sys+User) per algorithm (backend 4 KB)

Client-side CPU utilization approximately doubles under hybrid and post-quantum configurations, reflecting the additional cost of TLS initialization and key exchange preparation. Server-side CPU utilization shows a modest absolute variation (∼14–15%), but this must be interpreted in relative terms: hybrid configurations increase server CPU from 14.61% (x25519) to 15.46% (x25519_MLKEM768), representing a **+5.8% relative increase**. Pure MLKEM variants show slightly *lower* server CPU (13.91% for MLKEM512, −4.8%

relative), although the absolute differences are small and may fall within system-level variability. Although the server operates well below saturation at 100 TPS—a deliberate choice to avoid multithreading contention artifacts in the virtualized test environment—**extrapolating these relative differences to higher load levels suggests that PQC server-side overhead could become significant under production-scale traffic**.

Why the server-side cost is small. The modest server-side variation is expected rather than anomalous, and follows from the computational asymmetry of ML-KEM key exchange in TLS 1.3. In the ephemeral ML-KEM design used here (per the IETF hybrid key-exchange draft [9] and FIPS 203 [1]), the *client* generates a fresh ephemeral KEM key pair and transmits its public key in the ClientHello `key_share`, then decapsulates the shared secret on receipt of the ServerHello; the *server* performs only a single encapsulation against the client-supplied public key. Key generation and decapsulation—the heavier client-side operations—are precisely what surfaces in the TCP-to-TLS delay, whereas encapsulation is comparatively inexpensive. Importantly, the server does *not* generate a fresh, signed KEM key pair per handshake (as in KEMTLS-style proposals [19]); server authentication is provided by the unchanged RSA-2048 signature across all configurations, so the only added server work is the lightweight encapsulation. This confirms that the server was configured correctly and that the low observed cost reflects the genuine efficiency of ML-KEM encapsulation rather than a misconfiguration.

Network traffic scales linearly with key exchange size, from 32 bytes for x25519 up to 1568 bytes for MLKEM1024. Despite this increase, no proportional impact on latency is observed, indicating that the bandwidth overhead introduced by post-quantum key exchanges remains manageable in the evaluated architecture.

Implications for Different Client Types. The asymmetric CPU impact—where client-side utilization approximately doubles while the server shows a moderate but non-negligible relative increase of up to 5.8% for hybrids PQC —has important implications for PQC migration planning. On the client side, the overhead is substantial and may become a bottleneck for resource-constrained devices such as IoT sensors, mobile handsets, or embedded systems. On the server side, although absolute CPU differences are small ($\sim$1%), the relative impact is meaningful: at the tested load of 100 TPS the server operates at only $\sim$15% CPU, so a 1% absolute increase represents $\sim$6–7% of the utilized capacity. Under production workloads that drive servers closer to saturation, this relative overhead would translate into proportionally larger absolute increases.

More importantly when evaluating network devices that are acting as MiTM (Man-in-The-Middle) performing traffic decryption, inspection and then re-encryption of the traffic, the expected impact should be on the highest side. For such network devices a collaborative effect of both client side and server side PQC impact in addition to high TLS session rates can create the conditions of signifiant performance degradation when migrating to PQC encryption. Practitioners should therefore evaluate PQC overhead at client, server side, and

more importantly at network level with representative traffic load levels, before deployment.

4.9 End-to-End Perspective

The preceding per-layer analysis (Sections 4.1–4.8) demonstrated that PQC overhead is concentrated in the TCP-to-TLS delay (ClientHello construction), while the TLS handshake exchange, application, and TCP layers remain effectively algorithm-neutral. We now consolidate these findings into aggregate end-to-end connection times (TCP SYN to HTTP 200 OK), drawing on the stacked decomposition shown in Figure 3 and the per-layer data in Table 2.

Table 5 reports the end-to-end latency percentiles for each configuration, computed directly across all valid TCP streams in each capture (i.e., the true per-connection total time from TCP SYN to HTTP 200 OK, not the sum of per-layer percentiles, which would be statistically invalid since percentiles are not additive).

Table 5. End-to-end (E2E) latency (TCP SYN to HTTP 200 OK): percentiles and standard deviation per algorithmic configuration and backend response size. Values are computed directly from per-connection total times, not from summing per-layer percentiles.

Configuration	p50 (ms)	p95 (ms)	p99 (ms)	SD (ms)
x25519 (4 KB)	16.54	23.41	26.15	4.93
x25519 (40 KB)	19.88	27.10	30.37	30.87
x25519_MLKEM512 (4 KB)	20.26	26.91	29.70	5.70
x25519_MLKEM768 (4 KB)	19.63	26.14	28.30	6.42
x25519_MLKEM768 (40 KB)	22.25	29.89	33.40	21.17
MLKEM512 (4 KB)	18.92	25.53	28.08	6.15
MLKEM1024 (4 KB)	19.16	25.58	27.73	5.40

Key observations. Under the 4 KB backend scenario, the classical baseline (x25519) achieves a median end-to-end latency of 16.54 ms (SD = 4.93 ms). Hybrid configurations increase this to 20.26 ms (x25519_MLKEM512, +22.5%) and 19.63 ms (x25519_MLKEM768, +18.7%), while pure ML-KEM variants reach 18.92 ms (MLKEM512, +14.4%) and 19.16 ms (MLKEM1024, +15.8%). Notably, MLKEM1024 achieves virtually the same end-to-end latency as MLKEM512 despite its higher security level, consistent with the per-layer finding that the TLS handshake exchange is algorithm-neutral.

The absolute PQC overhead at the median ranges from 2.38 ms (pure ML-KEM) to 3.72 ms (hybrid) relative to x25519. As shown in Figure 3, this overhead is almost entirely attributable to the TCP-to-TLS layer (∼1.5 ms increase) with minor contributions from the TLS-to-App delay (∼0.4 ms). The TLS handshake and application layers contribute negligibly to the inter-algorithm differences. At the tail (p95), the overhead narrows: 2.12–3.50 ms across PQC configurations,

indicating that the penalty is relatively stable and does not amplify disproportionately under tail conditions.

For the 40 KB scenario, the standard deviations increase substantially (30.87 ms for x25519, 21.17 ms for x25519_MLKEM768), reflecting occasional long transfers at the application layer. Nevertheless, the absolute PQC overhead at the median remains stable at approximately 2.37 ms, and the relative overhead decreases from ∼19% (4 KB) to ∼12% (40 KB) due to the dilution effect described in Section 4.7.

5 Conclusions and Future Work

This paper presented a layered performance analysis of TLS 1.3 handshakes comparing classical (x25519), hybrid (x25519+ML-KEM), and pure post-quantum (ML-KEM) key exchange configurations under realistic load conditions of 100 requests per second. Through more than thirty experiments and statistical analysis of each protocol layer, we draw the following conclusions.

TCP layer independence (Section 4.2). The TCP handshake latency remains unaffected by the choice of cryptographic algorithm: median values range from 0.360 to 0.402 ms, a maximum difference of 0.042 ms ($\Delta = 0.17$, negligible). This confirms that PQC overhead does not propagate to lower protocol layers.

Client-side initialization as the dominant PQC cost (Section 4.3). The TCP-to-TLS delay (SYN-ACK to ClientHello) represents the most significant performance penalty introduced by post-quantum algorithms. Median latency increases from 0.294 ms (x25519, SD = 0.194 ms) to 1.73–1.90 ms for all PQC variants—a 6× increase ($\Delta \approx 7.7$, far exceeding Cohen's large-effect threshold of 0.8), making this the only layer where the PQC overhead is unambiguously significant. This cost is driven primarily by client-side key material generation and ClientHello construction, independent of backend response size and consistent across ML-KEM security levels.

TLS handshake exchange as an algorithm-neutral phase (Section 4.4). The TLS handshake exchange latency (ClientHello to Finished) shows no practically significant variation across algorithms: all configurations fall within 5.253–6.495 ms at the median, with differences of 0.1–0.9 ms yielding $\Delta = 0.03$–0.33 (negligible to small per Cohen's thresholds [24]). We stress that this neutrality concerns the on-the-wire message exchange, *not* the ClientHello construction, whose algorithm-sensitive cost is captured separately in the TCP-to-TLS layer. While pure ML-KEM microbenchmarks demonstrate faster encapsulation/decapsulation than ECDH scalar multiplication [20,18], this advantage is indistinguishable from noise in our TLS measurements. The practical implication is that pure PQC key exchange introduces *no measurable penalty in the handshake exchange* compared to the classical baseline at this load level (see Section 4.6 for normalized metrics).

Negligible application-layer impact (Section 4.5). Application response latency is entirely dominated by the end-to-end network path latency and response payload size (approximately 9 ms for 4 KB and 12 ms for 40 KB

responses). The maximum inter-algorithm variation (0.74 ms) yields $\Delta = 0.21$ (negligible to small), confirming no practically meaningful penalty attributable to the cryptographic algorithm. PQC migration does not degrade user-perceived application performance.

Moderate normalized cryptographic overhead (Section 4.6). Despite the ∼6× overhead in the TCP-to-TLS layer, the Cryptographic Overhead Share (COS) remains moderate: 6–14% of the total end-to-end connection time across all PQC configurations. Hybrid algorithms exhibit the highest COS (10.6–14.1%), while pure ML-KEM configurations achieve 6.0–7.9%. This confirms that the absolute PQC penalty, although concentrated in a single layer, has a limited impact on overall transaction latency.

Payload size dilution (Section 4.7). Increasing the backend response body from 4 KB to 40 KB does not affect the cryptographic layers (TCP-to-TLS, TLS handshake, TLS-to-App remain unchanged), while the application layer grows proportionally. Consequently, the relative PQC overhead decreases from ∼15% (4 KB) to ∼8% (40 KB) at the median, as the payload-dependent latency dilutes the fixed cryptographic cost.

CPU and network overhead (Section 4.8). Client-side CPU approximately doubles under PQC (from ∼3.7% to ∼8.2–8.7%), while server-side CPU remains within a narrow range (∼14–15%), with hybrid configurations showing up to ∼6% relative increase. Network traffic grows modestly from 5.2 to 6.6 Mb/s despite key exchange sizes increasing from 32 B (x25519) to 1568 B (MLKEM1024). PQC migration planning should consider client-side resource provisioning (especially for constrained devices), network devices, and server-side capacity at production load levels.

End-to-end perspective (Section 4.9). The PQC penalty on the end-to-end (E2E) transaction time is a *fixed* cost concentrated in connection establishment: PQC adds approximately 2–4 ms regardless of payload size, because the post-quantum cost is incurred once during ClientHello construction and is independent of the volume of application data subsequently transferred. The E2E time therefore depends on both the key-exchange algorithm *and* the payload size, but the post-quantum component does not grow with message size. Consequently, the *relative* weight of this fixed overhead shrinks as the payload (and thus the total transaction time) grows: in direct scenarios it amounts to roughly 15–20% of the E2E time, whereas in backend-proxied scenarios with larger cleartext responses it falls below 20% and continues to decrease with payload size.

Given the findings from this paper, there are few important feature research directions that are important to analyse in details including:

- extending the analysis to real network environments with commercial load balancers and MiTM (Man-in-The-Middle) inspection devices to quantify the performance impact when using PQC in TLS especially considering the high network load/connections these devices need to handle, where PQC performance impact may be signifiant.
- conducting stress tests beyond 100 TPS to identify the breaking point of PQC-enabled TLS under heavy load.

- evaluating post-quantum digital signature algorithms (Falcon, SPHINCS+, ML-DSA) and their impact on certificate verification latency.
- extending the comparison to further key-exchange algorithms and parameter sets beyond the five evaluated here—for example, additional ML-KEM security levels (e.g., MLKEM768 in pure mode), the SecP256r1MLKEM768 and SecP384r1MLKEM1024 hybrid groups, and alternative KEM families—to broaden the coverage of the post-quantum key-exchange design space.
- fine-grained profiling of the individual cryptographic operations within the TCP-to-TLS layer (ephemeral key-pair generation, encapsulation, and decapsulation) to attribute the client-side overhead to specific primitives and identify concrete targets for code-level optimization.
- isolating the resource cost (CPU and bandwidth) attributable *exclusively* to the TLS handshake, decoupled from subsequent application-record delivery, to characterize the handshake-only footprint of hybrid versus pure PQC configurations.
- evaluating PQC TLS overhead in stacks with native PQC support, such as Nginx with OpenSSL 3.5 and Locust with OpenSSL 3.5.

Acknowledgments

In accordance with the conference policy on AI-generated content, we disclose the following. An AI coding assistant (GitHub Copilot, powered by large language models) was used partially during two activities in this work: (1) *Script development*: the data-reduction tool (`pcap_layer_analysis.py`) was co-developed with AI assistance for code generation, refactoring, and debugging; and (2) *Paper drafting*: AI was used as a writing aid in the preparation of selected sections of this manuscript, including text generation, restructuring, and grammar refinement. In both cases, all AI-generated output was reviewed, validated, and edited by the authors, who bear full responsibility for the final content.

This work was supported by the PQNEXT project, which has received funding from the European Union's Horizon Europe research and innovation programme under Grant Agreement No. 101225759.

References

1. NIST. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, 2024. https://doi.org/10.6028/NIST.FIPS.203
2. NIST. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*, 2024. https://doi.org/10.6028/NIST.FIPS.204
3. NIST. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)*, 2024. https://doi.org/10.6028/NIST.FIPS.205
4. Open Quantum Safe Project. *liboqs: Open-Source C Library for Quantum-Safe Cryptographic Algorithms*. https://openquantumsafe.org/
5. Google Chromium Blog. *Post-Quantum Cryptography in Chrome*, 2024. https://blog.chromium.org/2024/04/post-quantum-cryptography-in-chrome.html

6. S. Ahmad, L. Valenta, and B. Westerbaan. *Cloudflare now uses post-quantum cryptography to talk to your origin server*, 2023. https://blog.cloudflare.com/post-quantum-to-origins/
7. A. Hopkins and M. Campagna. *Post-quantum TLS now supported in AWS KMS*, AWS Security Blog, 2019 (updated 2023/2024). https://aws.amazon.com/blogs/security/post-quantum-tls-now-supported-in-aws-kms/
8. OpenSSL Software Foundation. *OpenSSL Cryptography and SSL/TLS Toolkit*. https://www.openssl.org/
9. D. Stebila, S. Fluhrer, and S. Gueron. *Hybrid Key Exchange in TLS 1.3*. Internet-Draft, IETF, 2025. https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/
10. K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila. *Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3*. Internet-Draft (replaced by WG draft *draft-ietf-tls-ecdhe-mlkem*), IETF, 2024–2025. https://datatracker.ietf.org/doc/draft-kwiatkowski-tls-ecdhe-mlkem/
11. S. Sikeridis, P. Kampanakis, and M. Devetsikiotis. *Post-Quantum Authentication in TLS 1.3: A Performance Study*. Cryptology ePrint Archive, Report 2020/071. https://eprint.iacr.org/2020/071.pdf
12. M. Raabi et al. *Security and Performance Analyses of Post-Quantum Digital Signature Algorithms and Their TLS and PKI Integrations*. *Cryptography*, 9(2):38, 2025. https://www.mdpi.com/2410-387X/9/2/38
13. M. Sosnowski et al. *The Performance of Post-Quantum TLS 1.3*. In *Proc. ACM Internet Measurement Conference*, 2024. https://dl.acm.org/doi/abs/10.1145/3624354.3630585
14. P. Kampanakis et al. *The Impact of Data-Heavy, Post-Quantum TLS 1.3 on the Time-To-Last-Byte of Real-World Connections*. Cryptology ePrint Archive, Report 2024/176. https://eprint.iacr.org/2024/176.pdf
15. J. Zheng et al. *Faster Post-Quantum TLS 1.3 Based on ML-KEM: Implementation and Assessment*. In *Proc. ACNS*, LNCS, Springer, 2024. https://link.springer.com/chapter/10.1007/978-3-031-70890-9_7
16. Keysight Technologies. *CyPerf: Cloud-Native Application and Security Test Solution*. https://www.keysight.com/us/en/products/network-test/cloud-test/cyperf.html
17. National Security Agency. *Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) Cybersecurity Advisory*, updated release, 2025. https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF
18. C. Paquin, D. Stebila, and G. Tamvada. *Benchmarking Post-Quantum Cryptography in TLS*. In *Proc. ACNS*, LNCS, vol. 12147, Springer, 2020. https://eprint.iacr.org/2019/1447.pdf
19. P. Schwabe, D. Stebila, and T. Wiggers. *More Efficient Post-Quantum KEMTLS with Pre-distributed Public Keys*. In *Proc. ESORICS*, LNCS, vol. 14345, Springer, 2024. https://eprint.iacr.org/2021/779
20. J. Bos et al. *CRYSTALS – Kyber: A CCA-Secure Module-Lattice-Based KEM*. In *Proc. IEEE Euro S&P*, 2018. https://eprint.iacr.org/2017/634
21. D. J. Bernstein. *Introduction to post-quantum cryptography*. In *Post-Quantum Cryptography*, pp. 1–14, Springer, 2009. https://doi.org/10.1007/978-3-540-88702-7_1
22. D. Liu, S. I. Jang, N. Sultan, S. Lai, S. C. K. Chau, J. Chan, and H. Suzuki. *Post-Quantum Cryptography Migration of a Physical 5G Testbed*. In *Proc. QRSEC*, ACM, 2025. https://doi.org/10.1145/3733820.3764679

23. Telefonica. *Public_Performance_Quantum_Safe: Repository with the experimental setup and packet-capture analysis script.* GitHub repository. https://github.com/Telefonica/Public_Performance_Quantum_Safe
24. J. Cohen. *Statistical Power Analysis for the Behavioral Sciences.* 2nd ed., Lawrence Erlbaum Associates, 1988. https://doi.org/10.4324/9780203771587
25. G. V. Glass. *Primary, Secondary, and Meta-Analysis of Research. Educational Researcher*, 5(10):3–8, 1976. https://doi.org/10.3102/0013189X005010003